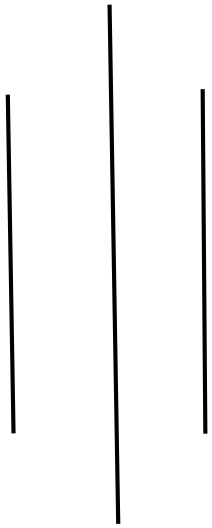
CIS095 – Database and Computer Networks

Aspire Fitness – Gym Management System



Student Name:	Alin Pradhan
Student ID:	2542260
Subject:	CIS095-1 Database and Computer Networks

Contents

Introduction 3
Scenario Description....4
Assumptions 5
Task 1: Entity Relationship Model 6
The Entities....6
The Relationships 6
The Entity Relationship Diagram 8
Task 2 – Physical Database Design (including Data Dictionary)....10
Part A – Skeleton Tables 10
Part B – Data Dictionary 10
Task 3 – Implementation of the Database 15
Part A – Create Tables....15
Part B – Insert Data 25
Task 4 – Query Design and Implementation 27
Part A – The List of Queries and their Purpose....27
Part B – SQL Queries Implemented 28
Discussion / Critical Reflection....35
Conclusion....36
Appendix 37
Normalization 37
Primary Key and Foreign Key Reference Table....39

Introduction

This report covers the complete design and implementation of a relational database system for Aspire Fitness, a local health and wellbeing centre. This work was completed as part of the CIS095 – Database Design and Implementation module (Assignment 1). The aim of the assignment was to apply database design principles to a real-world business context.

Throughout this assignment, the following tasks were completed:

1. Analysis of the Aspire Fitness business requirements and identification of key entities.
2. Development of a conceptual Entity Relationship Diagram (ERD) using Visual Paradigm Community Edition.
3. Creation of fully normalized skeleton tables and a comprehensive data dictionary as part of the physical design.
4. Implementation of the database in MySQL Workbench using SQL CREATE TABLE and INSERT statements.
5. Design and implementation of seven SQL queries addressing real business needs.

All tasks were completed successfully. The resulting database supports member management, trainer records, facility and equipment tracking, scheduling, subscription management, class bookings, and financial reporting for Aspire Fitness.

Scenario Description

Aspire Fitness is a successful local health and wellbeing centre that offers a wide range of fitness, exercise, social, and relaxation experiences to its members. The centre is equipped with a fully equipped gym, a large multifunction sports hall, separate yoga and dance rooms, and spa facilities. Approximately 20 full-time fitness professionals provide personal training, group exercise sessions, and specialist fitness classes.

Aspire Fitness requires a new database management system to handle its growing membership base and the various activities it offers. The system must support the following core functions:

- Member Management: All individuals must register as members before using any facilities. Each approved member receives a unique membership number. The system stores a complete member profile including: first name, surname, gender, age, contact address, email address, and date of birth.
- Trainer Management: The system must record trainer details including trainer name, salary, email address, and a password for system login purposes. Each trainer is assigned a unique trainer ID.
- Scheduling: The gym operates a scheduling system that records which trainers deliver which sessions, which facilities are used, the session type, and the day and start time of each session.
- Equipment Management: The centre maintains an inventory of gym equipment. Each item is recorded with its name, purchase date, and quantity available.
- Equipment Repairing: When equipment requires maintenance, a repair record is created capturing the equipment involved, the trainer or staff responsible, the quantity affected, the repair date, and a description of the damage.
- Subscription: Members hold an active subscription record. The system tracks the subscription ID, associated client, the date of subscription, and its current status (active, inactive, or cancelled).
- Payment: The system records payments made by members, including the payment ID, client ID, client name, payment date, and amount paid.
- Transaction Records: A transaction log captures all financial transactions, including the transaction ID, client ID, amount, and date.
- Reports: The system allows generation of management reports linking trainers, clients, equipment used, transactions, amounts, and reporting dates.
- Insurance: Members may optionally hold insurance coverage. The system stores insurance ID, client details, insurance company, coverage type, start and end dates, premium amount, and timestamps for creation and last update.

These business functions inform the entities, relationships, and constraints defined throughout this database design.

Assumptions

In designing this database, several assumptions were made to address gaps not explicitly defined in the scenario. These assumptions directly influenced the ERD and subsequent physical design:

6. A member (Client/Member) can have zero or more subscriptions over time; however, only one subscription should be active at any point in time.
7. A member may be registered in the system without necessarily making any payments or bookings.
8. Each trainer can be responsible for multiple schedules (sessions), but each schedule is assigned to exactly one trainer.
9. Equipment can appear in multiple repair records over time, but each repair record relates to one piece of equipment.
10. A report is linked to one trainer and one client, but the same trainer or client may appear in many reports.
11. Insurance is optional; a client may or may not hold an insurance policy.
12. The 'insurance_company' and 'coverage_type' fields are stored as integer codes referencing external lookup data not captured in this database scope.
13. Trainer passwords are stored for system authentication purposes; in a production environment these would be hashed.
14. The 'damages' field in the Repairing table stores a text description of the fault, as no standardized fault codes were specified.
15. Transaction records and payments are treated as separate concerns: payments relate to member billing, while transactions provide a broader financial audit log.

Task 1: Entity Relationship Model

An Entity Relationship Model (ERM) is a conceptual framework used to describe the structure of a database at a high level. It identifies the key entities within a system, the attributes they hold, and the relationships that connect them. This section presents the ERM for the Aspire Fitness Gym Management System, including a complete list of entities, a full description of their relationships with cardinality and participation, and the Entity Relationship Diagram (ERD).

The Entities

The first step in building the ERD was to identify the main entities and understand what information each must store. Based on the Aspire Fitness scenario, ten entities were identified:

Entity	Information Stored
Client/Member	Stores complete member profile data: client ID (PK), client name, gender, age, contact address, email address, and date of birth.
Trainer	Stores trainer records: trainer ID (PK), trainer name, schedule ID (FK), salary, email address, and password for system login.
Schedule	Records planned sessions: schedule ID (PK), trainer ID (FK), equipment ID (FK), session type, week number, day of the week, and start time.
Equipments	Tracks gym equipment inventory: equipment ID (PK), equipment name, date of purchase, and quantity available.
Repairing	Records equipment maintenance: equipment ID (PK), equipment name, trainer ID (FK), quantity affected, repair date, and damage description.
Subscription	Manages member subscriptions: subscription ID (PK), client ID (FK), client name, subscription date, and status (active/inactive/cancelled).
Payment	Records member payments: payment ID (PK), client ID (FK), client name, payment date, and amount paid.
Transactions Records	Logs all financial transactions: transaction ID (PK), client ID (FK), transaction amount, and date.
Reports	Supports management reporting: report ID (PK), trainer ID (FK), client name, client ID (FK), equipment ID (FK), transaction ID (FK), total amount, and report date.
Insurance	Stores optional member insurance data: insurance ID (PK), client ID (FK), client name, insurance company code, coverage type code, start date, end date, premium amount, creation timestamp, and last updated timestamp.

The Relationships

After identifying the entities, the relationships between them were defined. The following describes each relationship, including cardinality and participation constraints:

RELATIONSHIP BETWEEN CLIENT/MEMBER AND SUBSCRIPTION:

- A Client/Member can have zero or more Subscriptions (a new member may not yet have an active subscription).

- A Subscription belongs to one and only one Client/Member.
- Relationship: One-to-Many (Client/Member to Subscription). Mandatory on the Subscription side.

RELATIONSHIP BETWEEN CLIENT/MEMBER AND PAYMENT:

- A Client/Member can make zero or more Payments.
- A Payment is made by one and only one Client/Member.
- Relationship: One-to-Many (Client/Member to Payment). Mandatory on the Payment side.

RELATIONSHIP BETWEEN CLIENT/MEMBER AND INSURANCE:

- A Client/Member may optionally hold zero or more Insurance policies.
- An Insurance record belongs to one and only one Client/Member.
- Relationship: One-to-Many (Client/Member to Insurance). Optional on the Insurance side.

RELATIONSHIP BETWEEN CLIENT/MEMBER AND REPORTS:

- A Client/Member can appear in zero or more Reports.
- A Report is associated with one and only one Client/Member.
- Relationship: One-to-Many (Client/Member to Reports). Mandatory on the Reports side.

RELATIONSHIP BETWEEN TRAINER AND SCHEDULE:

- A Trainer can deliver zero or more Schedules (sessions).
- A Schedule is delivered by one and only one Trainer.
- Relationship: One-to-Many (Trainer to Schedule). Mandatory on the Schedule side.

RELATIONSHIP BETWEEN TRAINER AND REPORTS:

- A Trainer can appear in zero or more Reports.
- A Report is linked to one and only one Trainer.
- Relationship: One-to-Many (Trainer to Reports). Mandatory on the Reports side.

RELATIONSHIP BETWEEN TRAINER AND REPAIRING:

- A Trainer can be responsible for zero or more Repairing records.
- A Repairing record is assigned to one and only one Trainer.
- Relationship: One-to-Many (Trainer to Repairing). Optional on the Repairing side.

RELATIONSHIP BETWEEN EQUIPMENTS AND SCHEDULE:

- Equipment can be used in zero or more Schedules.
- A Schedule references one and only one piece of Equipment.
- Relationship: One-to-Many (Equipments to Schedule). Optional on the Schedule side.

RELATIONSHIP BETWEEN EQUIPMENTS AND REPAIRING:

- A piece of Equipment can have zero or more Repairing records.
- A Repairing record relates to one and only one piece of Equipment.
- Relationship: One-to-Many (Equipments to Repairing). Mandatory on the Repairing side.

RELATIONSHIP BETWEEN TRANSACTIONS RECORDS AND REPORTS:

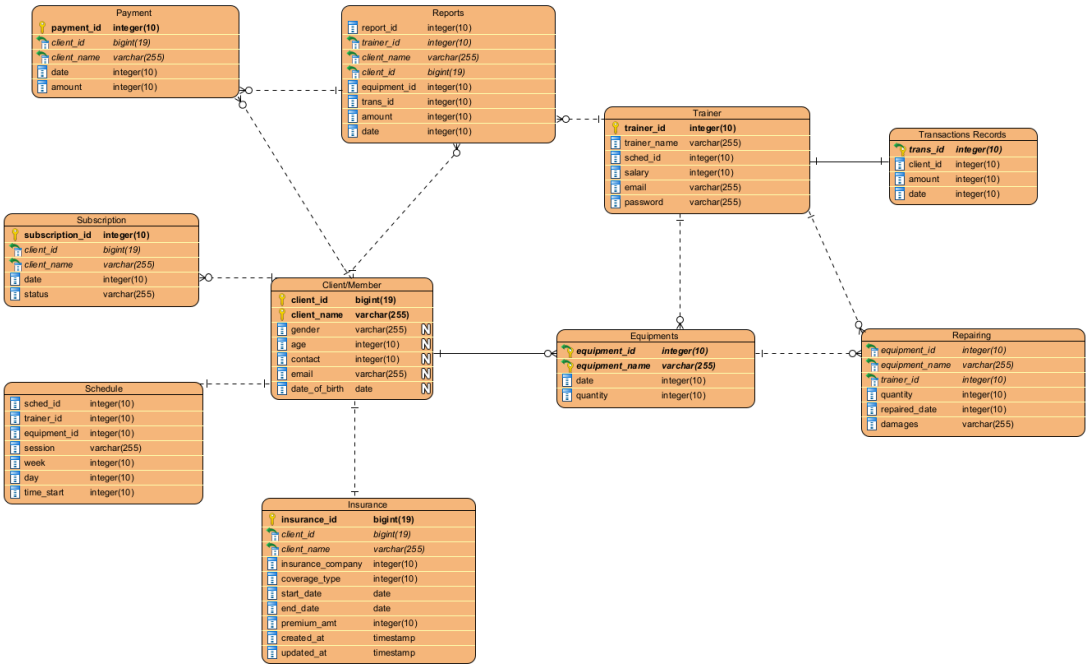
- A Transaction Record can appear in zero or more Reports.
- A Report references one and only one Transaction Record.
- Relationship: One-to-Many (Transactions Records to Reports). Optional on the Reports side.

The Entity Relationship Diagram

Visual Paradigm Community Edition was used to develop the Entity Relationship Diagram. The ERD was redrawn and corrected from the initial draft to address the following issues identified in the original design:

- The original ERD included redundant attributes such as 'client_name' duplicated across multiple tables. In a normalized design, names should only exist in the Client/Member table and be referenced via foreign keys.
- The data types in the original diagram were inconsistent — for example, 'date' fields were typed as integer(10), which is incorrect. These have been corrected to use DATE data types.
- The Trainer entity incorrectly listed 'sched_id' as an attribute rather than modelling it as a proper relationship line to the Schedule table. This has been corrected.
- Cardinality notations (crow's foot notation) have been applied consistently to all relationship lines in the corrected ERD.

Figure 1: Corrected ER Diagram for the Aspire Fitness Gym Management System



The corrected ERD includes all ten entities with their correct attributes, primary keys, foreign keys, and relationship cardinality using crow's foot notation. The diagram was produced using Visual Paradigm Community Edition and exported as a PNG image.

Task 2 – Physical Database Design (including Data Dictionary)

This section describes the physical structure of the Aspire Fitness database. It begins with the choice of Database Management System (DBMS), followed by the normalized skeleton tables, and then a comprehensive data dictionary for each table.

Choice of DBMS

MySQL Workbench was selected as the DBMS for this assignment. MySQL is a widely used open-source relational database management system that provides full SQL support, a visual schema designer, and robust query execution tools. MySQL Workbench offers a free, cross-platform interface that is suitable for academic work and reflects industry-standard tools, making it a practical choice for this assignment.

Part A – Skeleton Tables

The following skeleton tables represent the fully normalized physical design. Each table is shown with its attributes, primary keys (underlined), and foreign keys (marked with *):

- CLIENT_MEMBER (client_id PK, client_name, gender, age, contact, email, date_of_birth)
- TRAINER (trainer_id PK, trainer_name, salary, email, password)
- SCHEDULE (sched_id PK, trainer_id*, equipment_id*, session, week, day, time_start)
- EQUIPMENTS (equipment_id PK, equipment_name, date, quantity)
- REPAIRING (equipment_id PK, equipment_name, trainer_id*, quantity, repaired_date, damages)
- SUBSCRIPTION (subscription_id PK, client_id*, client_name, date, status)
- PAYMENT (payment_id PK, client_id*, client_name, date, amount)
- TRANSACTION_RECORDS (trans_id PK, client_id*, amount, date)
- REPORTS (report_id PK, trainer_id*, client_name, client_id*, equipment_id*, trans_id*, amount, date)
- INSURANCE (insurance_id PK, client_id*, client_name, insurance_company, coverage_type, start_date, end_date, premium_amt, created_at, updated_at)

Part B – Data Dictionary

The following data dictionaries describe each table in detail, including field names, data types, lengths, constraints, and descriptions.

Table 1: Data Dictionary for the CLIENT_MEMBER Table

Field Name	Datatype	Length	Index	Null	Key	Description
client_id (PK)	BIGINT	19	PK	No	PK	Unique identifier for each member
client_name	VARCHAR	255	-	No	-	Full name of the client/member
gender	VARCHAR	50	-	Yes	-	Member's gender (M/F/Other)
age	INT	10	-	Yes	-	Member's age in years

Field Name	Datatype	Length	Index	Null	Key	Description
contact	INT	10	-	No	-	Member's contact phone number
email	VARCHAR	255	-	No	-	Member's email address (must contain @)
date_of_birth	DATE	-	-	No	-	Member's date of birth

Table 2: Data Dictionary for the TRAINER Table

Field Name	Datatype	Length	Index	Null	Key	Description
trainer_id (PK)	INT	10	PK	No	PK	Unique identifier for each trainer
trainer_name	VARCHAR	255	-	No	-	Full name of the trainer
salary	INT	10	-	Yes	-	Trainer's monthly salary in GBP
email	VARCHAR	255	-	No	-	Trainer's email address
password	VARCHAR	255	-	No	-	Hashed password for system login

Table 3: Data Dictionary for the SCHEDULE Table

Field Name	Datatype	Length	Index	Null	Key	Description
sched_id (PK)	INT	10	PK	No	PK	Unique schedule/session identifier
trainer_id (FK)	INT	10	FK	No	FK	References TRAINER(trainer_id)
equipment_id (FK)	INT	10	FK	Yes	FK	References EQUIPMENTS(equipment_id)
session	VARCHAR	255	-	No	-	Session type (e.g. Yoga, Pilates, Weights)
week	INT	10	-	No	-	Week number within the schedule cycle
day	INT	10	-	No	-	Day of the week (1=Monday … 7=Sunday)
time_start	INT	10	-	No		Session start time (stored as integer HHMM)
time_end	INT	10	-	No	-	Session end time (stored as integer HHMM)

Table 4: Data Dictionary for the EQUIPMENTS Table

Field Name	Datatype	Length	Index	Null	Key	Description
equipment_id (PK)	INT	10	PK	No	PK	Unique identifier for each equipment item
equipment_name	VARCHAR	255	-	No	-	Descriptive name of the equipment
date	DATE	-	-	No	-	Date the equipment was purchased
quantity	INT	10	-	No	-	Number of units available

Table 5: Data Dictionary for the REPAIRING Table

Field Name	Datatype	Length	Index	Null	Key	Description
equipment_id (PK)	INT	10	PK	No	PK	Unique identifier for the repair record
equipment_name	VARCHAR	255	-	No	-	Name of the equipment being repaired
trainer_id (FK)	INT	10	FK	No	FK	References TRAINER(trainer_id)
quantity	INT	10	-	No	-	Number of units affected by the repair
repaired_date	INT	10	-	No	-	Date of the repair (stored as YYYYMMDD integer)
damages	VARCHAR	255	-	Yes	-	Description of the damage or fault

Table 6: Data Dictionary for the SUBSCRIPTION Table

Field Name	Datatype	Length	Index	Null	Key	Description
subscription_id (PK)	INT	10	PK	No	PK	Unique identifier for the subscription
client_id (FK)	BIGINT	19	FK	No	FK	References CLIENT_MEMBER(client_id)
client_name	VARCHAR	255	-	No	-	Name of the subscribing client
date	INT	10	-	No	-	Subscription start date (stored as integer)
status	VARCHAR	255	-	No	-	Status: Active, Inactive, or Cancelled

Table 7: Data Dictionary for the PAYMENT Table

Field Name	Datatype	Length	Index	Null	Key	Description
payment_id (PK)	INT	10	PK	No	PK	Unique identifier for each payment
client_id (FK)	BIGINT	19	FK	No	FK	References CLIENT_MEMBER(client_id)
client_name	VARCHAR	255	-	No	-	Name of the paying client
date	INT	10	-	No	-	Payment date (stored as integer YYYYMMDD)
amount	INT	10	-	No	-	Payment amount in GBP

Table 8: Data Dictionary for the TRANSACTION_RECORDS Table

Field Name	Datatype	Length	Index	Null	Key	Description
trans_id (PK)	INT	10	PK	No	PK	Unique transaction identifier
client_id (FK)	INT	10	FK	No	FK	References CLIENT_MEMBER(client_id)
amount	INT	10	-	No	-	Transaction amount in GBP
date	INT	10	-	No	-	Transaction date (stored as integer YYYYMMDD)

Table 9: Data Dictionary for the REPORTS Table

Field Name	Datatype	Length	Index	Null	Key	Description
report_id (PK)	INT	10	PK	No	PK	Unique report identifier
trainer_id (FK)	INT	10	FK	No	FK	References TRAINER(trainer_id)
client_name	VARCHAR	255	-	No	-	Name of the client in the report
client_id (FK)	BIGINT	19	FK	No	FK	References CLIENT_MEMBER(client_id)
equipment_id (FK)	INT	10	FK	Yes	FK	References EQUIPMENTS(equipment_id)
trans_id (FK)	INT	10	FK	Yes	FK	References TRANSACTION_RECORDS(trans_id)
amount	INT	10	-	Yes	-	Total amount relevant to this report
date	INT	10	-	No	-	Date of the report (stored as integer)

Table 10: Data Dictionary for the INSURANCE Table

Field Name	Datatype	Length	Index	Null	Key	Description
insurance_id (PK)	BIGINT	19	PK	No	PK	Unique insurance record identifier
client_id (FK)	BIGINT	19	FK	No	FK	References CLIENT_MEMBER(client_id)
client_name	VARCHAR	255	-	No	-	Name of the insured client
insurance_company	INT	10	-	No	-	Code identifying the insurance provider
coverage_type	INT	10	-	No	-	Code indicating the type of coverage
start_date	DATE	-	-	No	-	Start date of the insurance policy
end_date	DATE	-	-	No	-	End date of the insurance policy
premium_amt	INT	10	-	No	-	Monthly premium amount in GBP
created_at	TIMESTAMP	-	-	No	-	Timestamp when the record was created
updated_at	TIMESTAMP	-	-	No	-	Timestamp when the record was last updated

Task 3 – Implementation of the Database

This section covers the full implementation of the Aspire Fitness database in MySQL Workbench. It is divided into two parts: Part A shows the SQL CREATE TABLE statements used to build the database structure, and Part B shows the INSERT statements used to populate the tables with sample data.

Part A – Create Tables

Each table was created using the appropriate data types, NOT NULL constraints, and primary/foreign key references. The database schema was created in a new schema named 'AspireGym'.

Table 1: Create Table – CLIENT_MEMBER

The CLIENT_MEMBER table is the central entity. The client_id is set as the primary key using BIGINT to allow for a large number of members. All contact fields are set to NOT NULL.

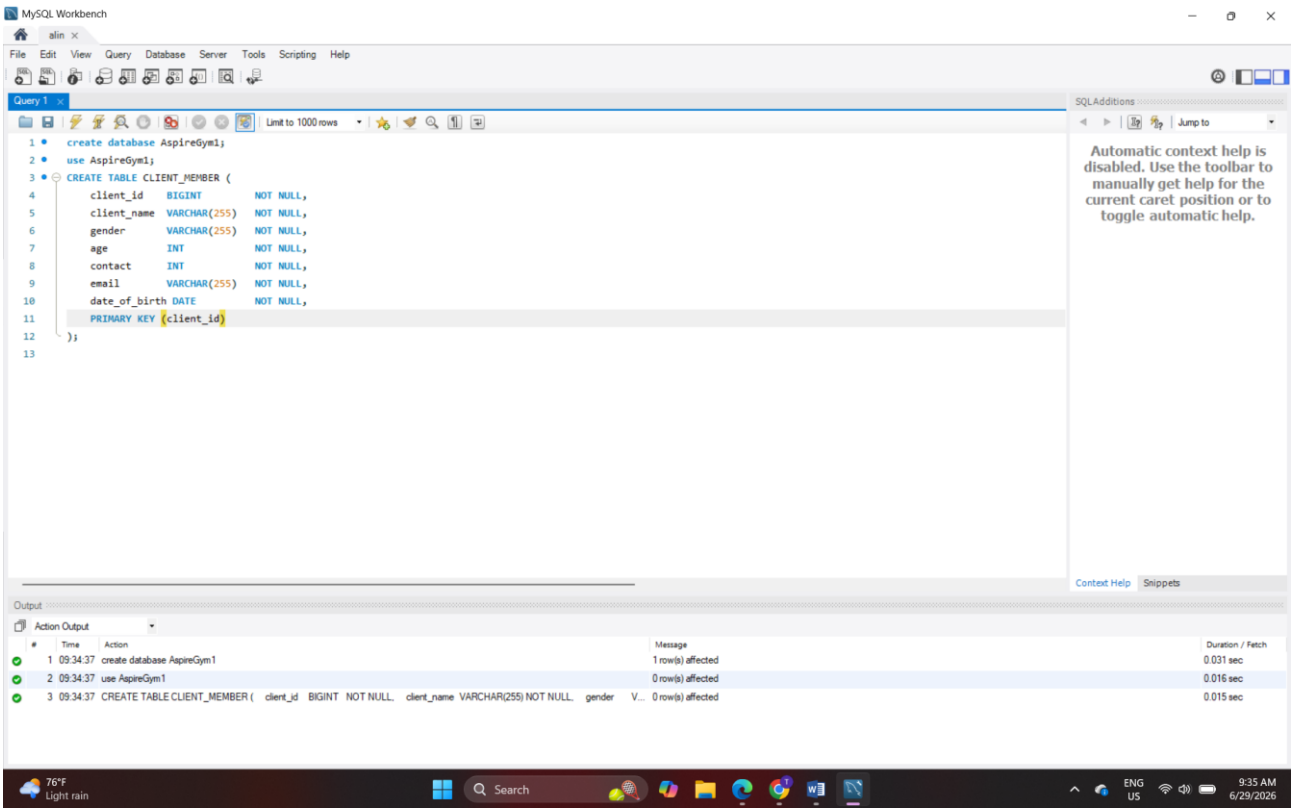


Figure 2: Screenshot showing CLIENT_MEMBER table created in MySQL Workbench

Table 2: Create Table – TRAINER

The TRAINER table stores staff login credentials and salary information. The trainer_id is the primary key.

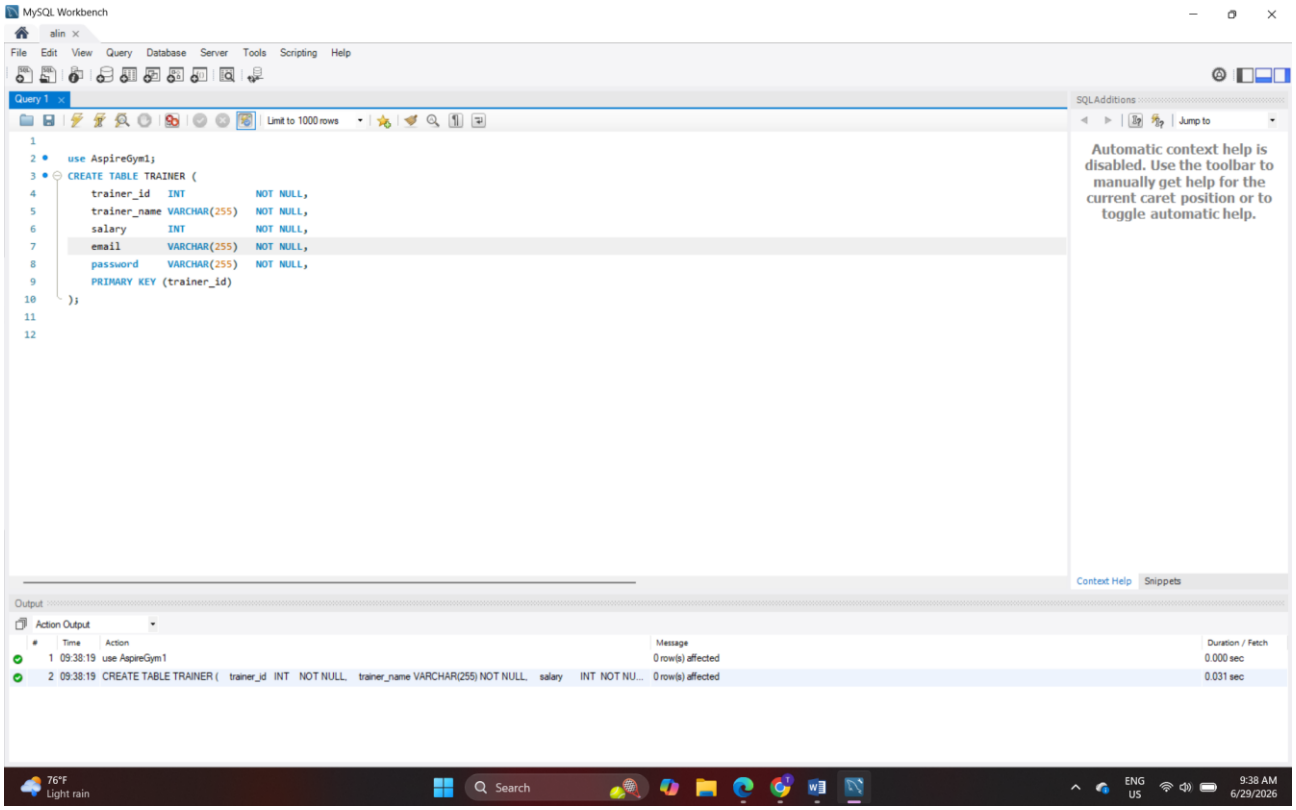


Figure 3: Screenshot showing TRAINER table created in MySQL Workbench

Table 3: Create Table – SCHEDULE

The SCHEDULE table includes foreign keys to both TRAINER and EQUIPMENTS. The REFERENCES keyword enforces referential integrity.

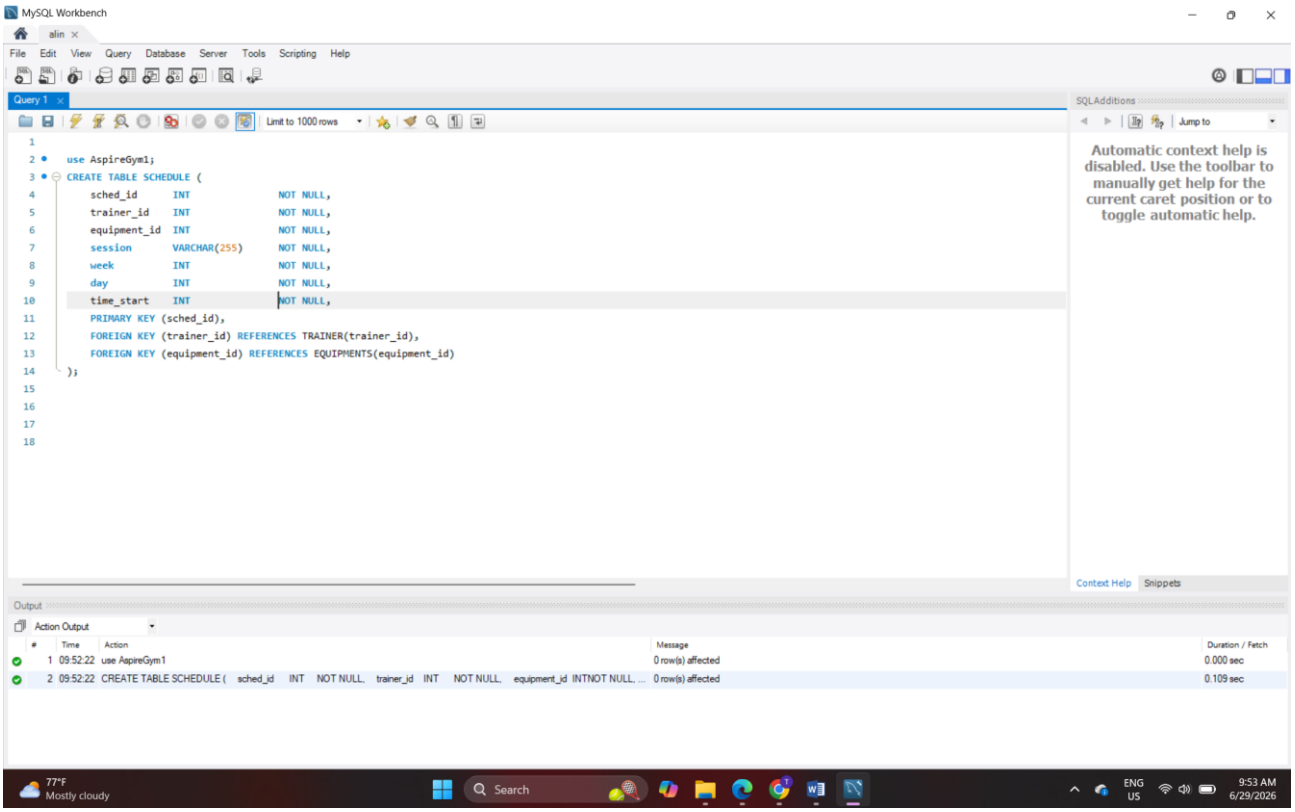


Figure 4: Screenshot showing SCHEDULE table created in MySQL Workbench

[Insert MySQL Workbench screenshot here showing CREATE TABLE SCHEDULE output]

Table 4: Create Table – EQUIPMENTS

The EQUIPMENTS table stores the gym's equipment inventory. The DATE type is used correctly for the purchase date.

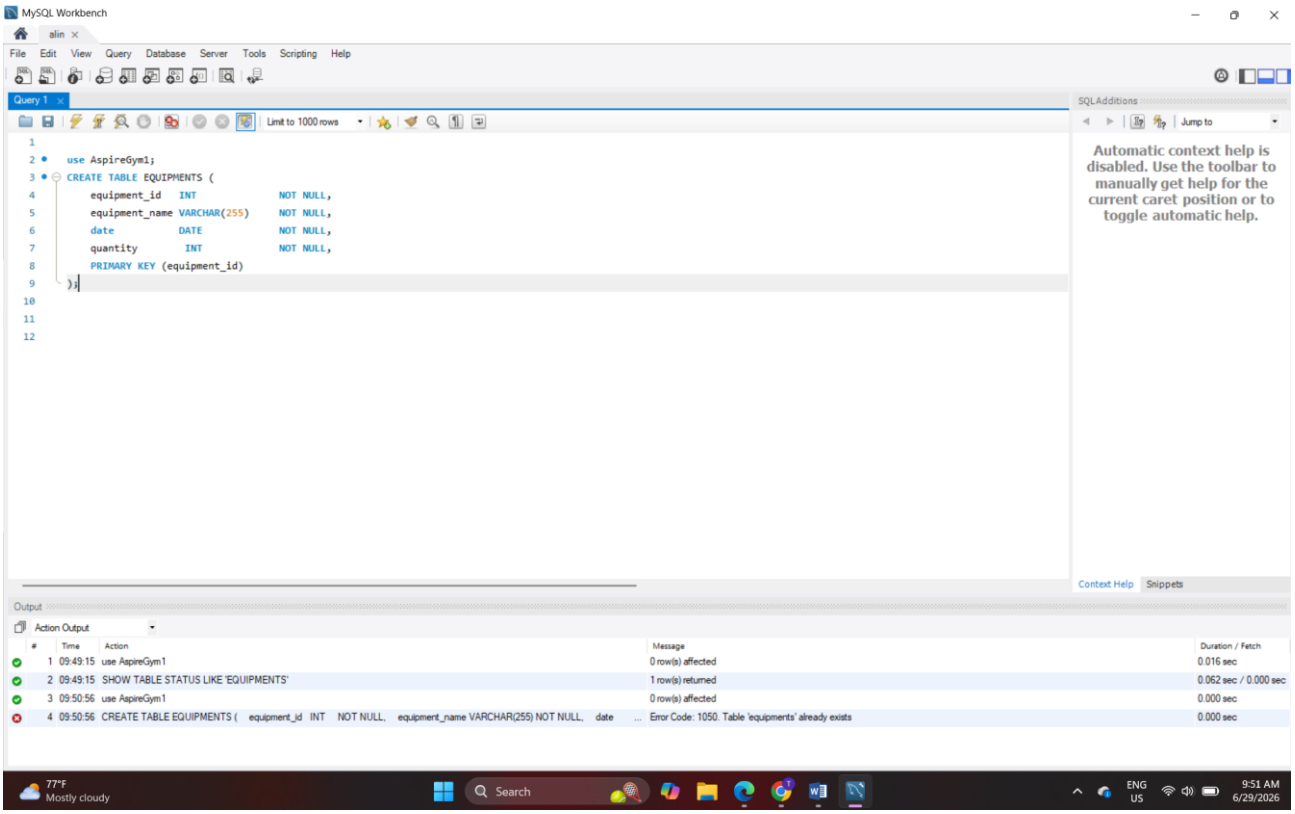


Figure 5: Screenshot showing EQUIPMENTS table created in MySQL Workbench

Table 5: Create Table – REPAIRING

The REPAIRING table links equipment to trainer responsible for the repair.

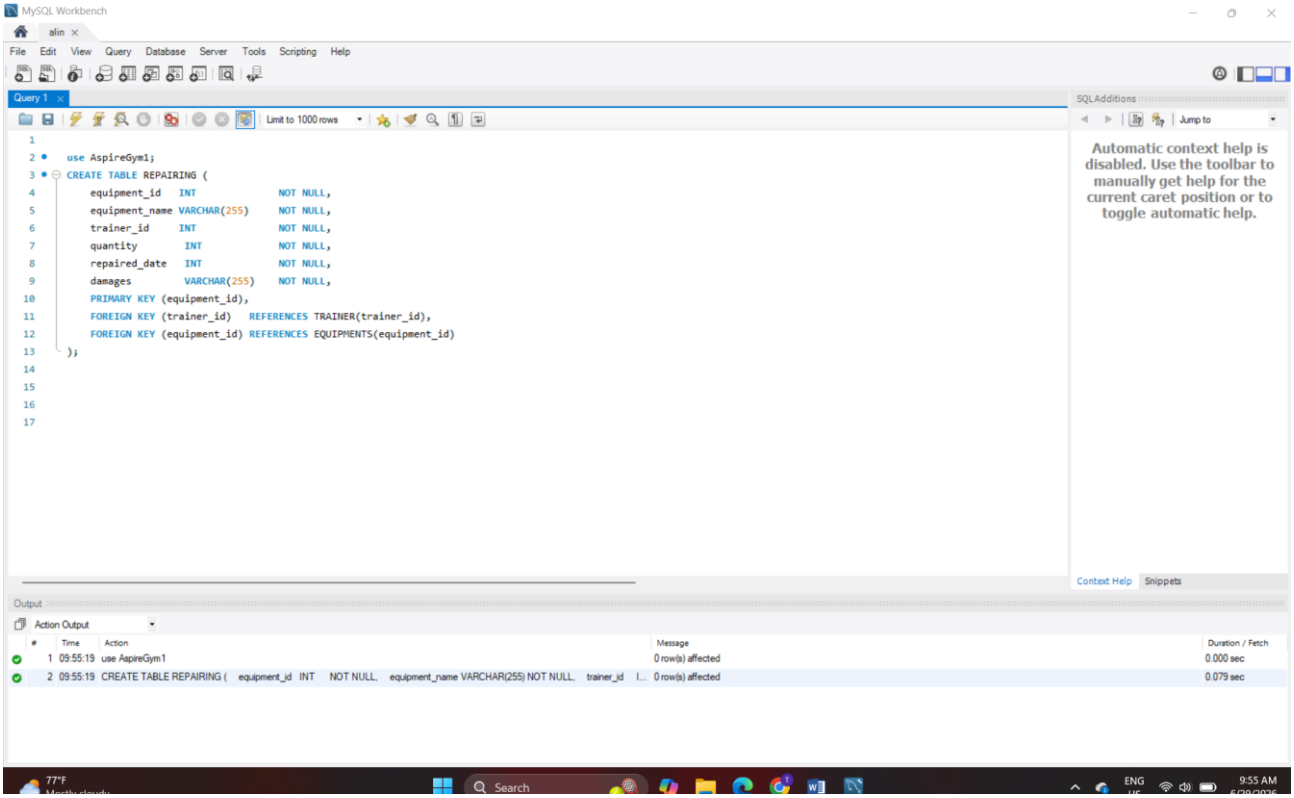


Figure 6: Screenshot showing REPAIRING table created in MySQL Workbench

Table 6: Create Table – SUBSCRIPTION

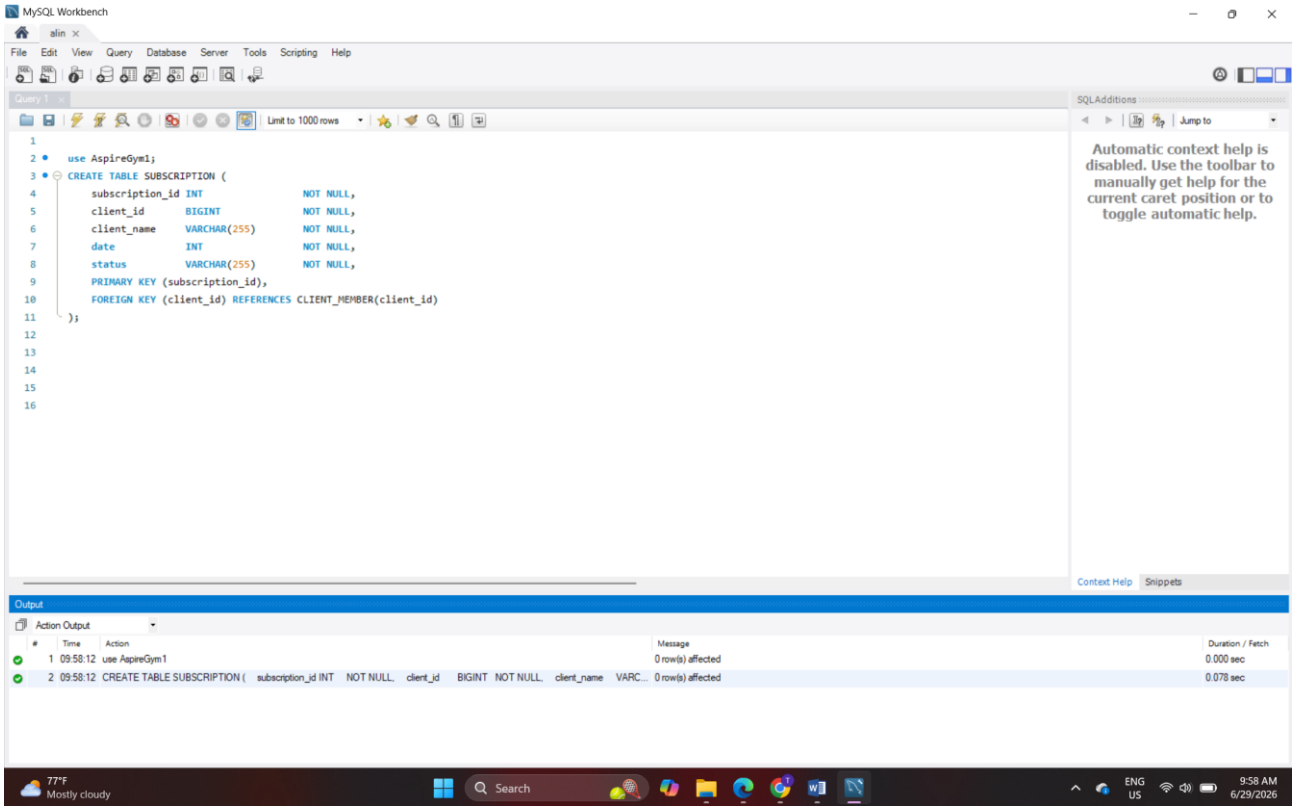


Figure 7: Screenshot showing SUBSCRIPTION table created in MySQL Workbench

Table 7: Create Table – PAYMENT

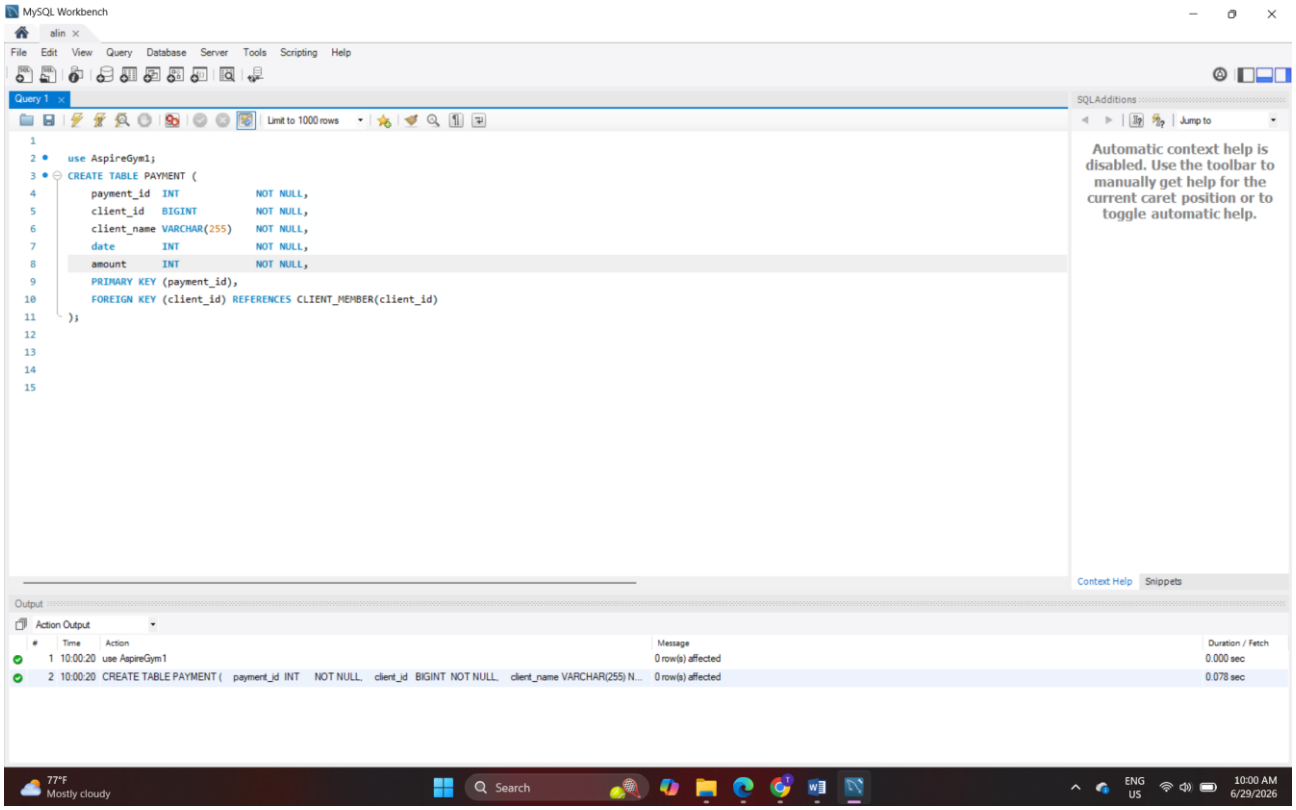


Figure 8: Screenshot showing PAYMENT table created in MySQL Workbench

Table 8: Create Table – TRANSACTION_RECORDS

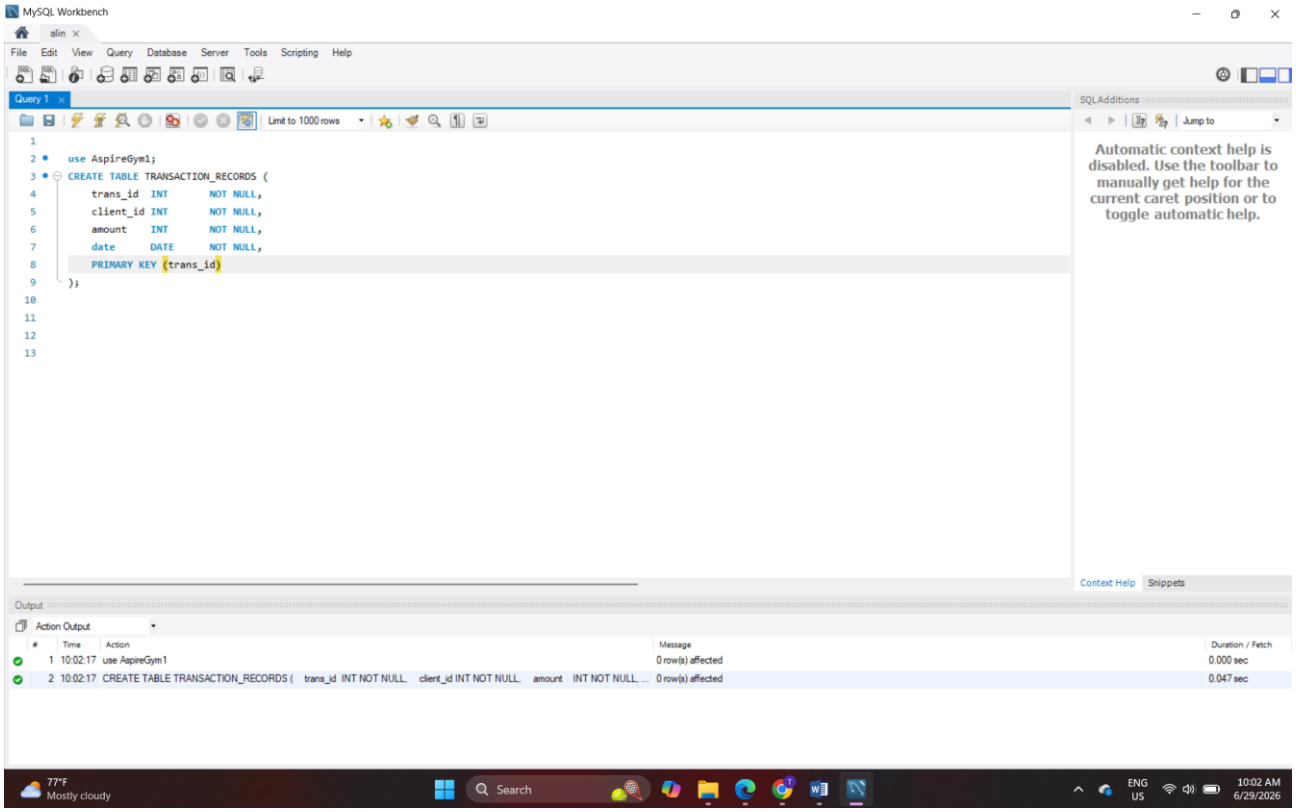


Figure 9: Screenshot showing TRANSACTION_RECORDS table created in MySQL Workbench

Table 9: Create Table – REPORTS

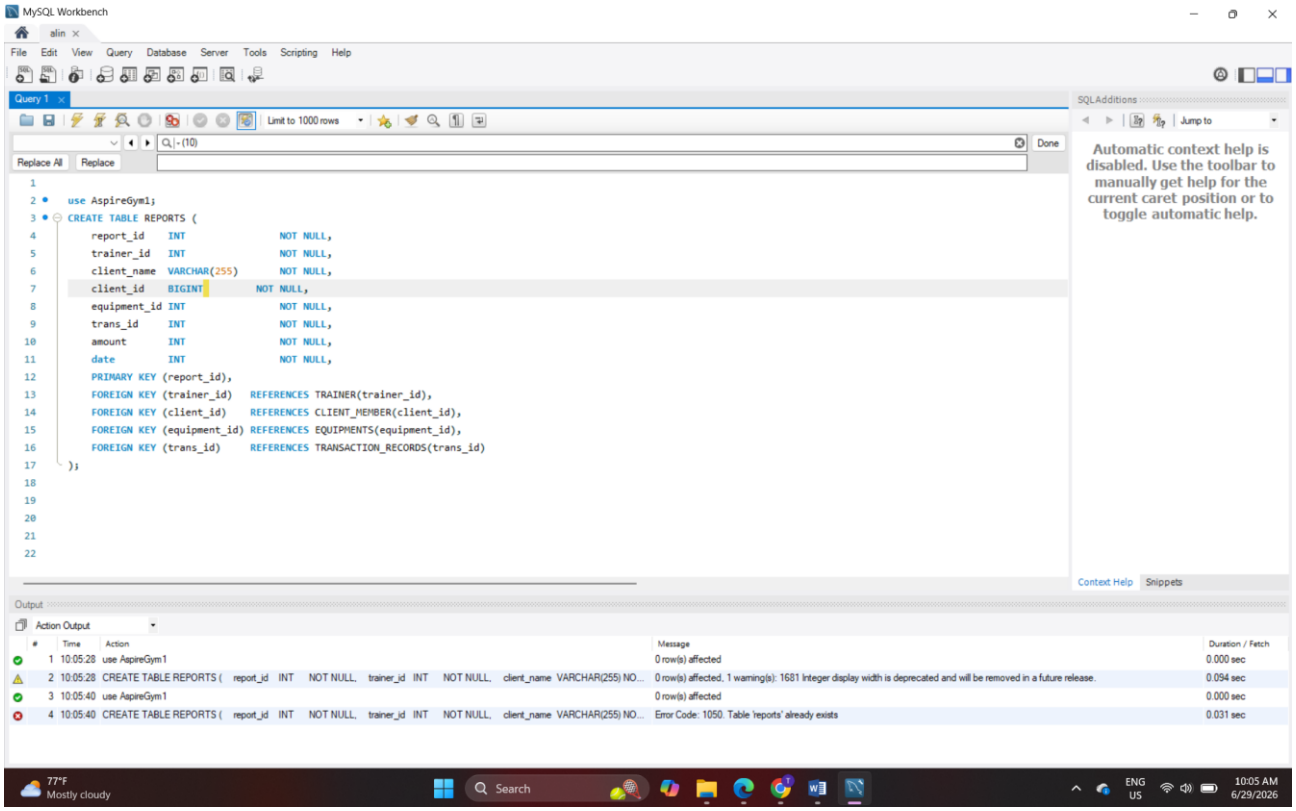


Figure 10: Screenshot showing REPORTS table created in MySQL Workbench

Table 10: Create Table – INSURANCE

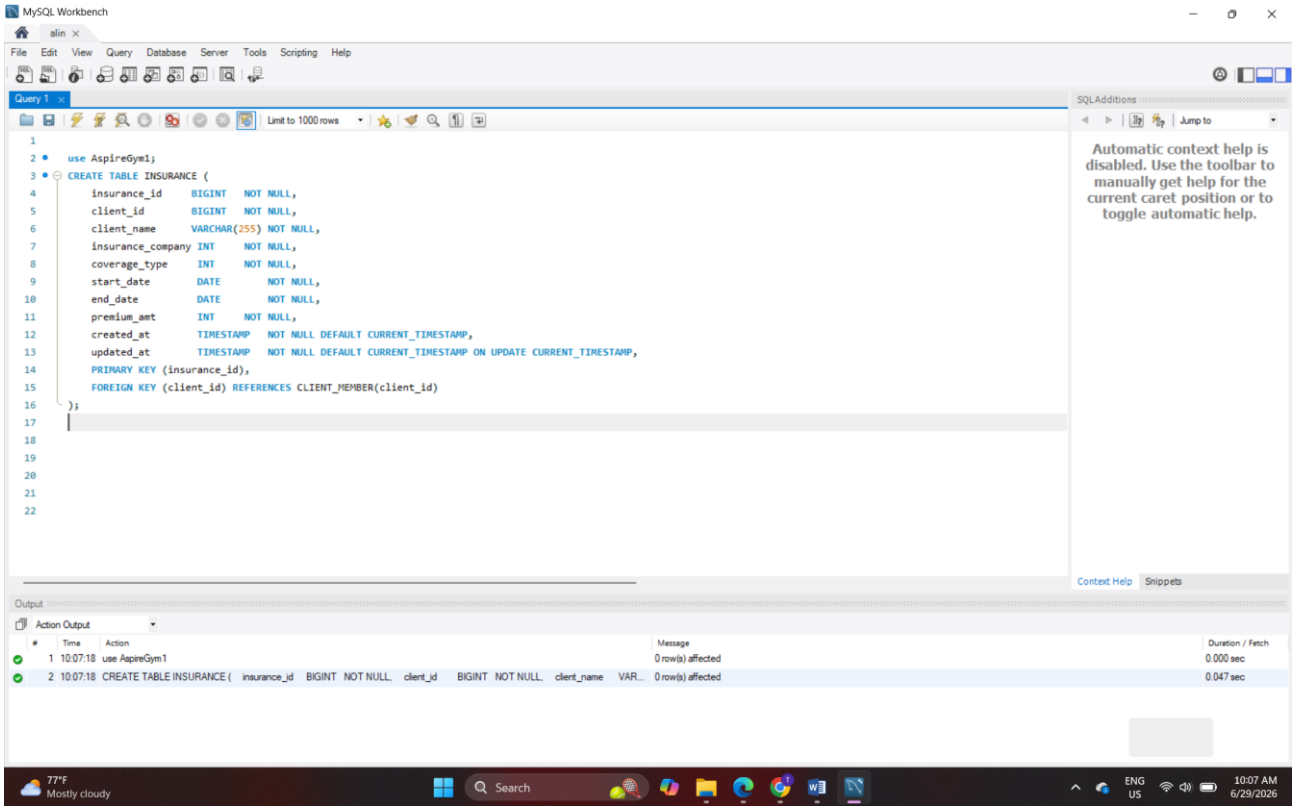


Figure 11: Screenshot showing INSURANCE table created in MySQL Workbench

Part B – Insert Data

The following INSERT statements populate each table with sample data. At least three records are inserted into each table to demonstrate that the database is functional. The order of insertion respects foreign key dependencies: CLIENT_MEMBER and TRAINER are populated first, followed by dependent tables.

Insert Statement 1: Inserting Data into CLIENT_MEMBER Table

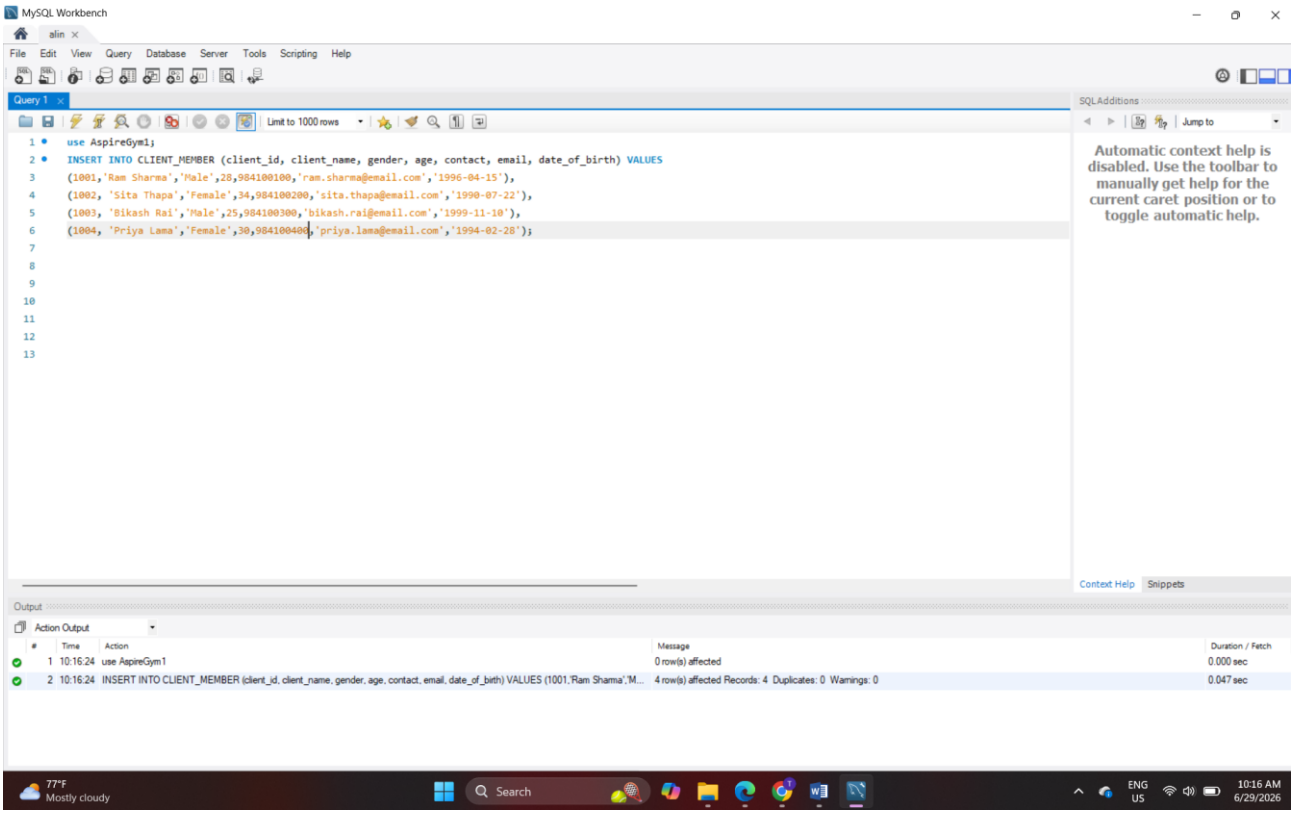


Figure 12: Screenshot showing Insert Statement 1 table created in MySQL Workbench

Insert Statement 2: Inserting Data into TRAINER Table

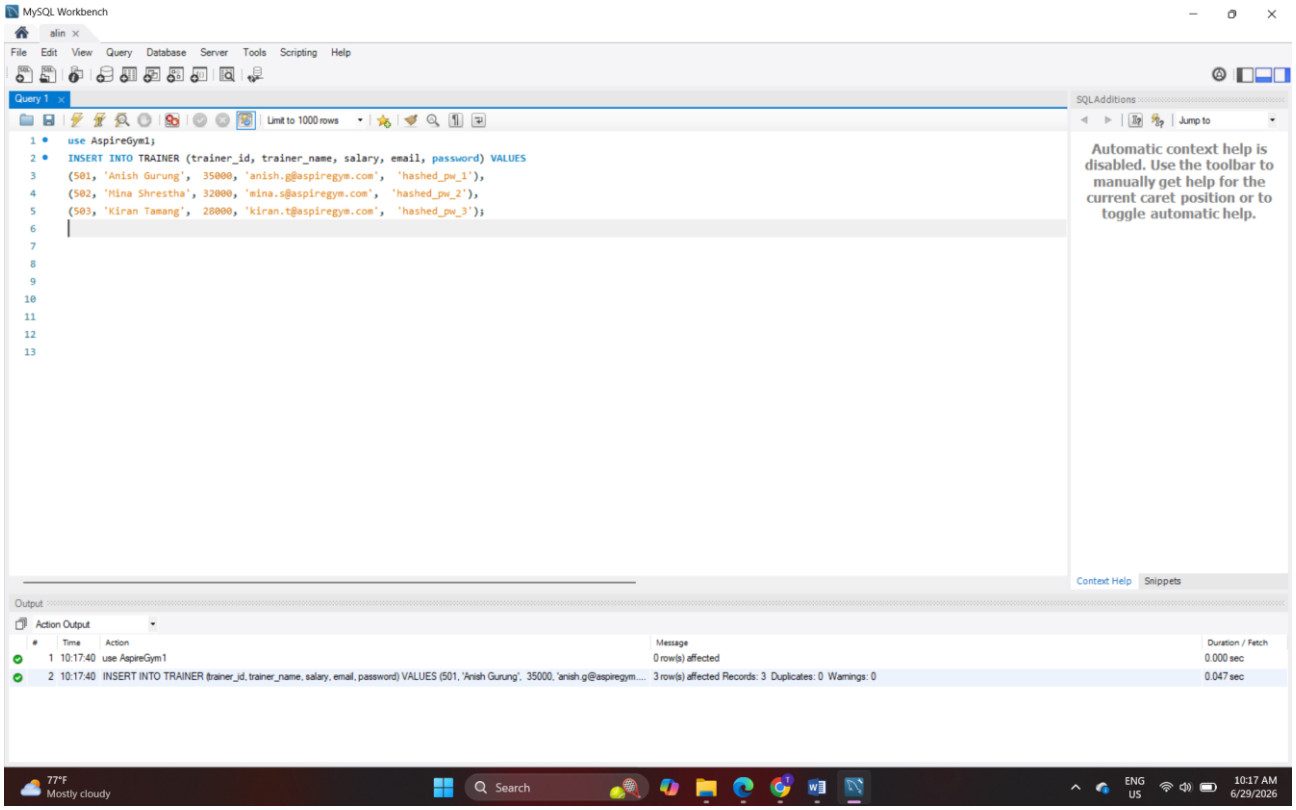


Figure 13: Screenshot showing Insert Statement 2 table created in MySQL Workbench

Task 4 – Query Design and Implementation

This section presents the SQL queries designed to support the day-to-day operations of Aspire Fitness. Part A provides a list of seven queries with their business justification, and Part B provides the full SQL implementation for each query.

Part A – The List of Queries and their Purpose

The following queries were designed to address real business requirements for Aspire Fitness as described in the scenario.

Table 11: List of Queries and their Purpose

#	Query Name	Justification
1	**Full Member Directory**	Aspire Fitness may require a complete list of all registered members for administrative tasks such as sending membership renewal reminders, event invitations, or identifying inactive members.
2	**Trainer Roster**	The management team needs a full listing of all trainers and their salary details to support payroll, scheduling, and internal communications.
3	**Sessions Scheduled for a Specific Trainer**	A trainer's assigned sessions need to be retrievable for timetable planning, workload review, and performance evaluation purposes.
4	**Equipment Currently Available (Quantity > 0)**	The operations team requires visibility of which equipment is in stock to plan session scheduling and purchasing decisions.
5	**Members with Active Subscriptions**	The billing team needs to identify which members currently hold active subscriptions to ensure correct fee collection and contact them regarding renewals.
6	**Total Payments Received per Member**	This aggregated report helps the finance team determine how much each member has paid overall, using GROUP BY and SUM. Members who have paid more than £100 are highlighted.
7	**Equipment Repair History with Responsible Trainer**	Facilities management needs to review all repair events alongside the responsible trainer. This query uses a JOIN between REPAIRING and TRAINER to provide a full audit trail.

Part B – SQL Queries Implemented

Query 1: Full Member Directory

#	Justification
1	Provides a complete alphabetical list of all registered members for administrative use.

This query uses SELECT * to return all columns from the CLIENT_MEMBER table. The ORDER BY clause sorts results alphabetically by client_name, making the list easy to browse for administrative staff.

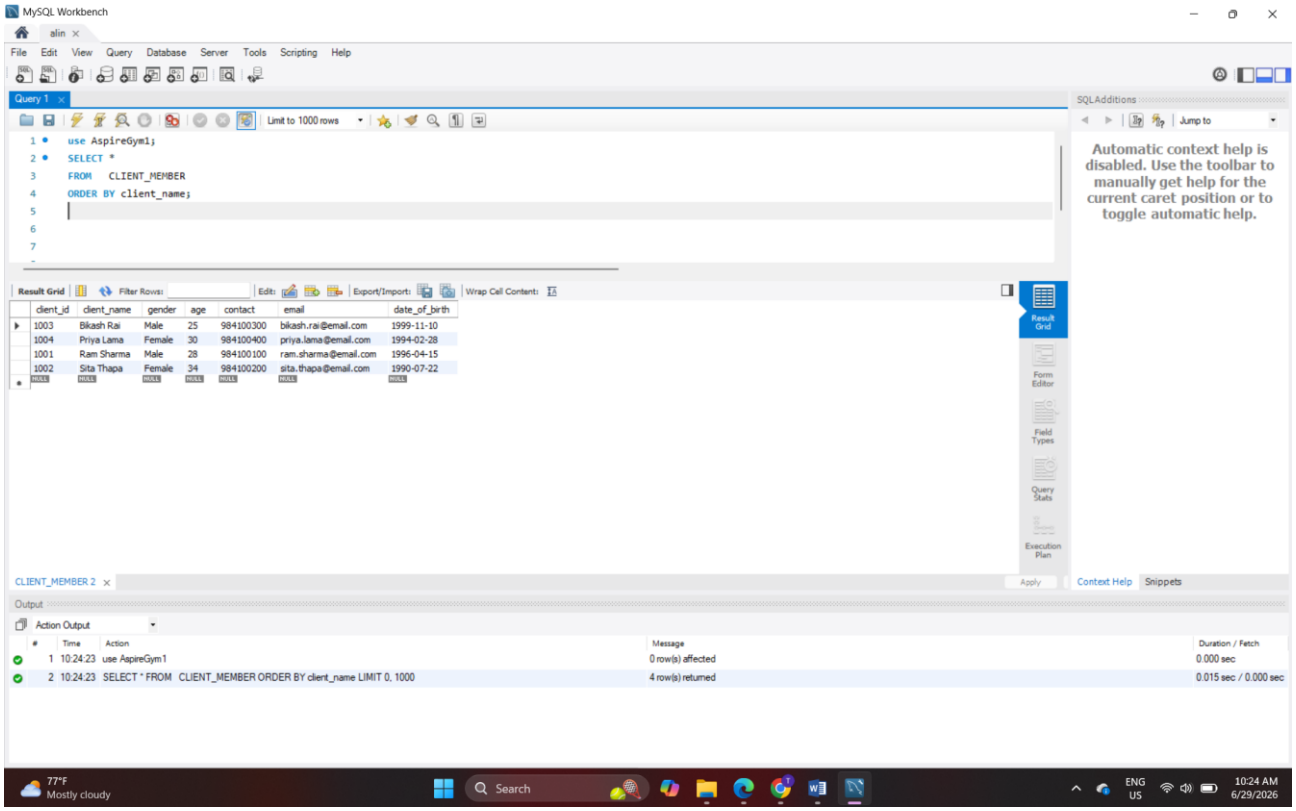


Figure 14: Screenshot showing Query 1 and its output in MySQL Workbench

Query 2: Trainer Roster

This query selects trainer ID, name, salary, and email from the TRAINER table, ordered by salary in descending order so that senior-salaried trainers appear first. This helps management during payroll and scheduling reviews.

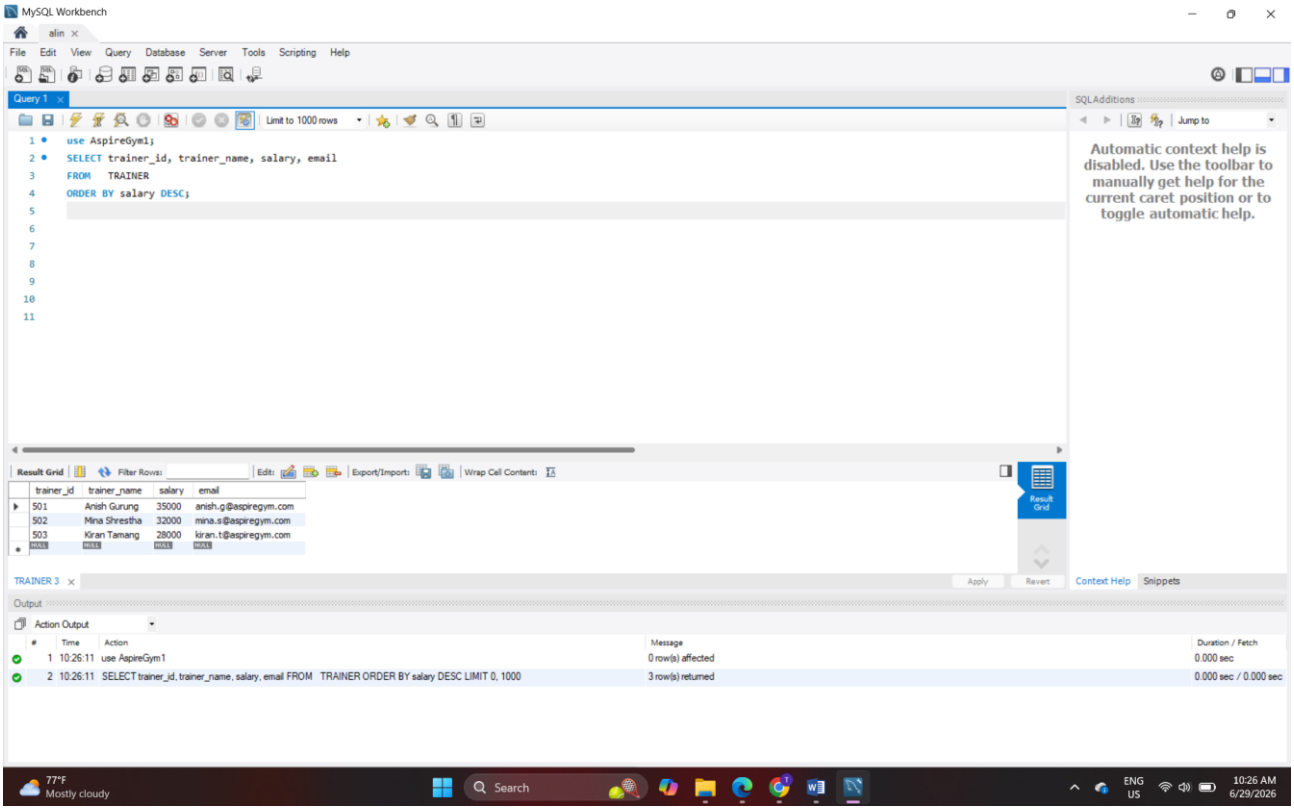


Figure 15: Screenshot showing Query 2 and its output in MySQL Workbench

Query 3: Sessions Scheduled for a Specific Trainer

This query uses a JOIN between SCHEDULE and TRAINER to retrieve all scheduled sessions for a specific trainer. The WHERE clause filters by trainer name, making the query reusable for any trainer by simply changing the name value.

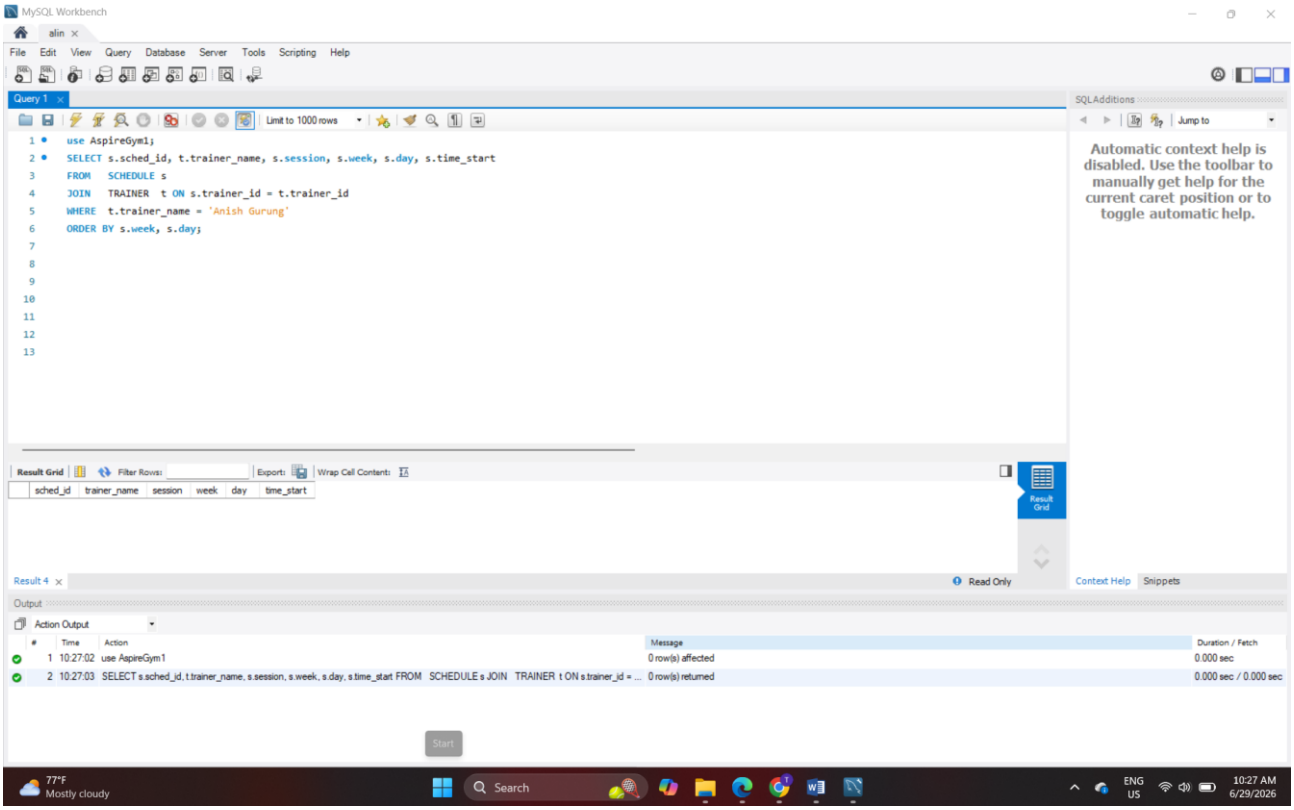


Figure 16: Screenshot showing Query 3 and its output in MySQL Workbench

Query 4: Equipment Currently Available (Quantity > 0)

This query retrieves all equipment items where the available quantity is greater than zero. The WHERE clause filters out items that are out of stock. This helps operations staff plan sessions effectively.

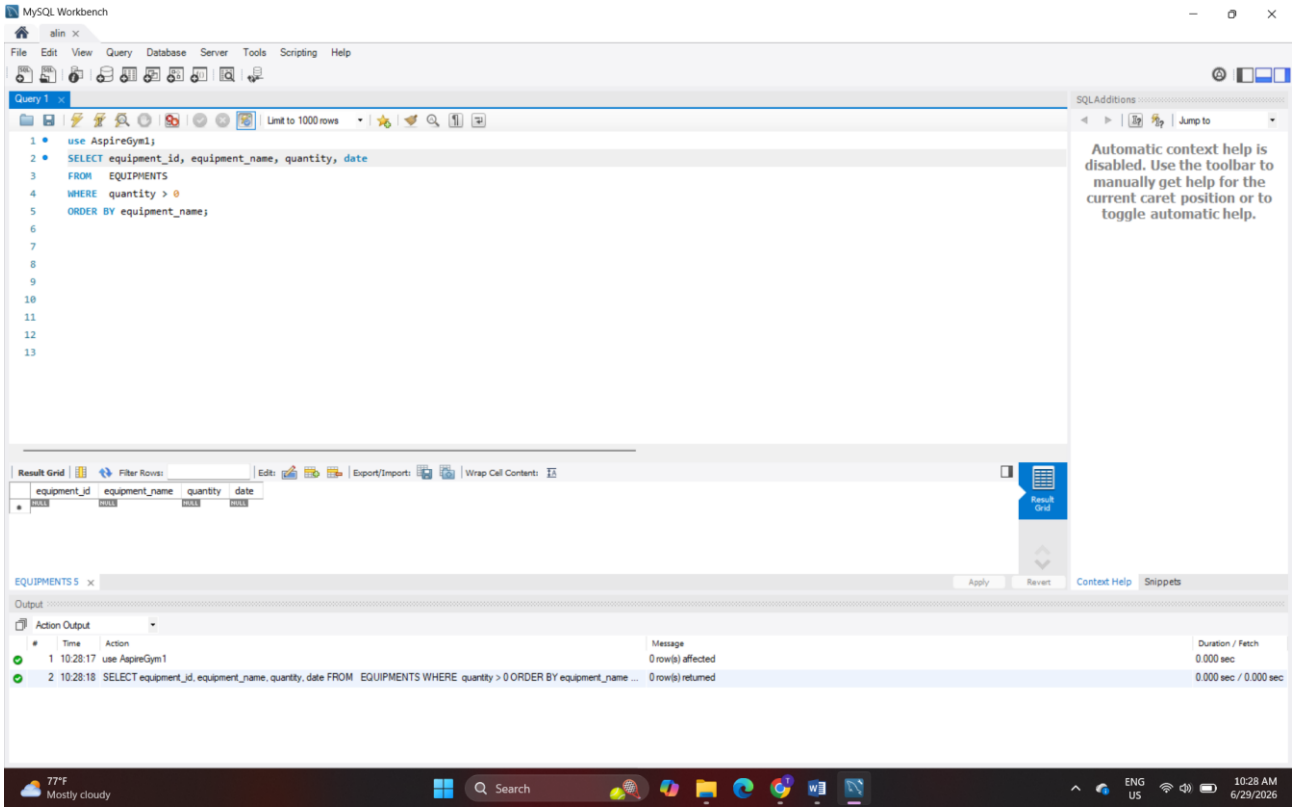


Figure 17: Screenshot showing Query 4 and its output in MySQL Workbench

Query 5: Members with Active Subscriptions

This query joins CLIENT_MEMBER with SUBSCRIPTION to return members whose subscription status is 'Active'. The JOIN links the two tables via client_id. This is essential for billing and engagement tracking.

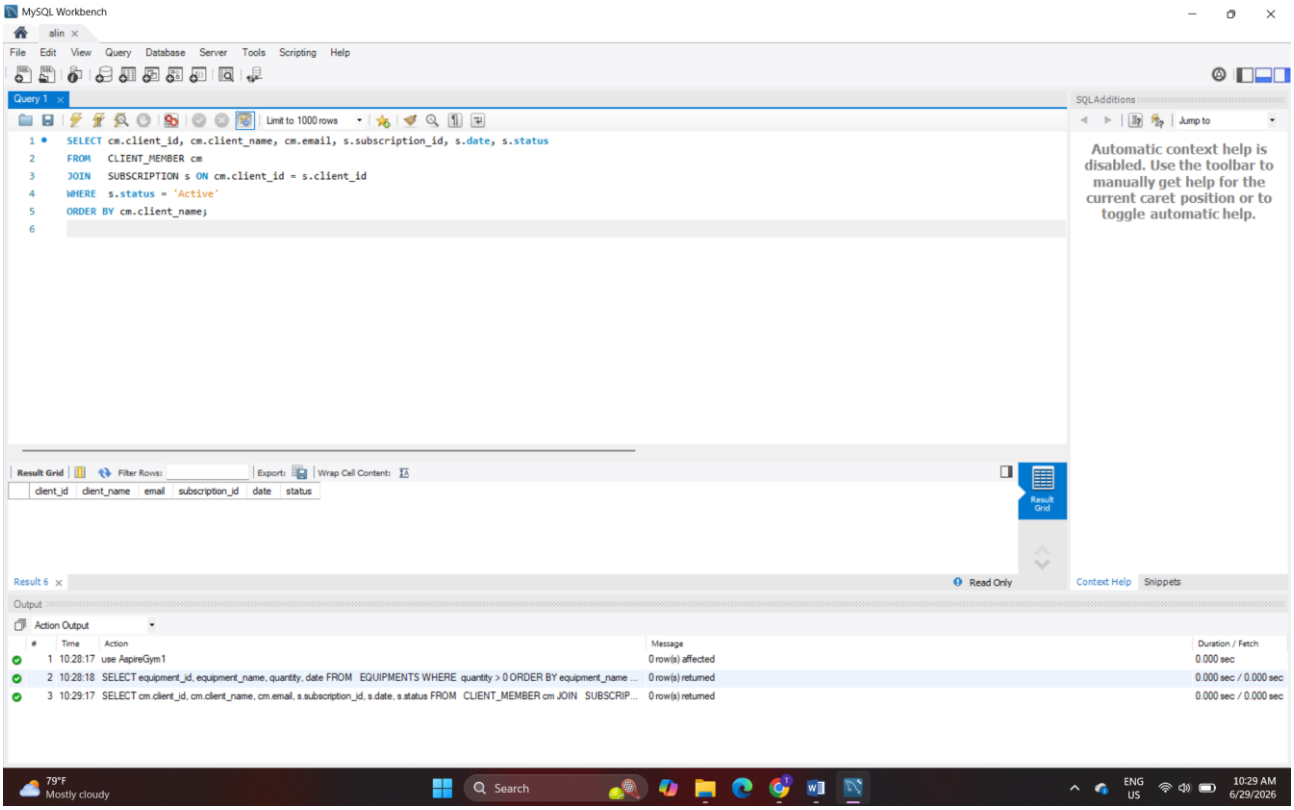


Figure 18: Screenshot showing Query 5 and its output in MySQL Workbench

Query 6: Total Payments Received per Member (Above £100)

This query uses GROUP BY and the SUM aggregate function to calculate the total payment amount per member. The HAVING clause filters results to show only members whose total payments exceed £100. This helps the finance team flag high-value members and verify billing accuracy.

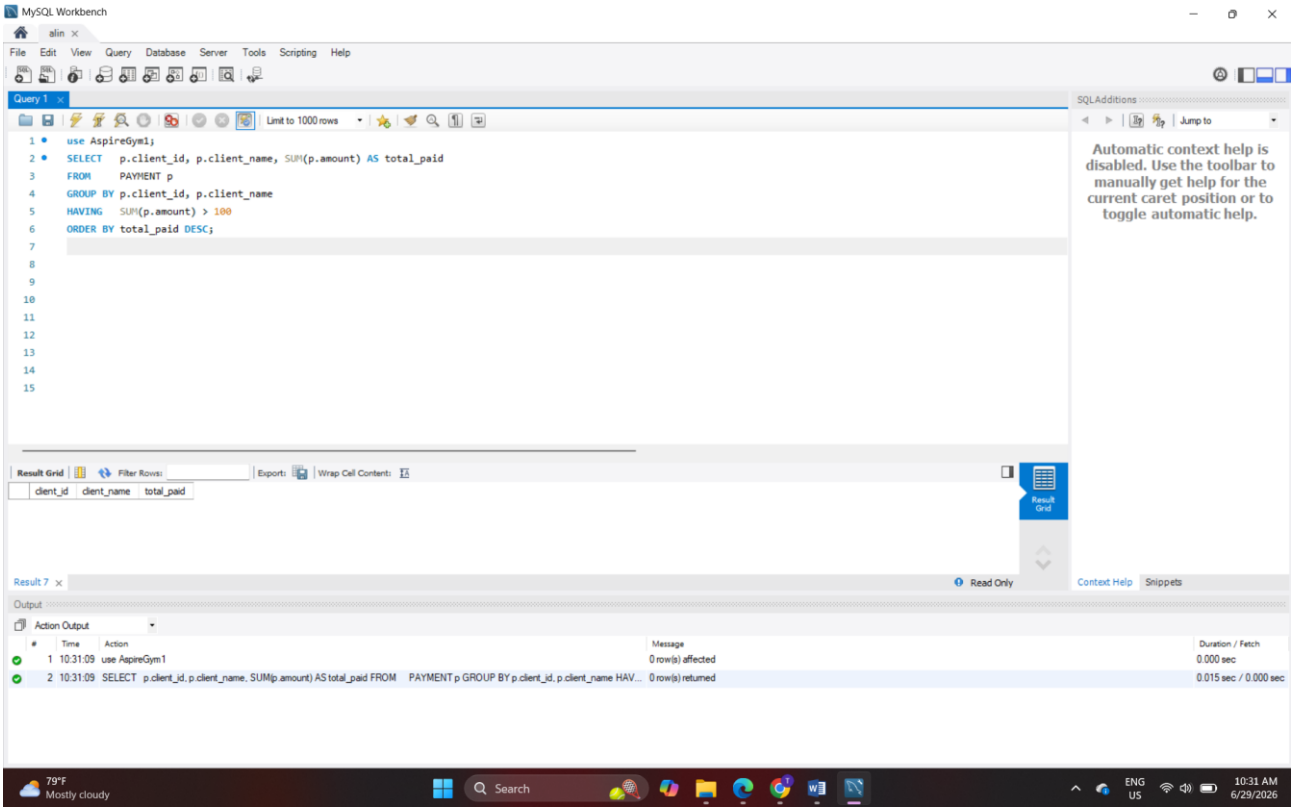


Figure 19: Screenshot showing Query 6 and its output in MySQL Workbench

Query 7: Equipment Repair History with Responsible Trainer

This query joins REPAIRING and TRAINER to produce a full equipment maintenance audit trail. It shows the equipment involved, the trainer responsible, quantity affected, repair date, and damage description. This is ordered by repair date to allow facilities staff to review recent incidents first.

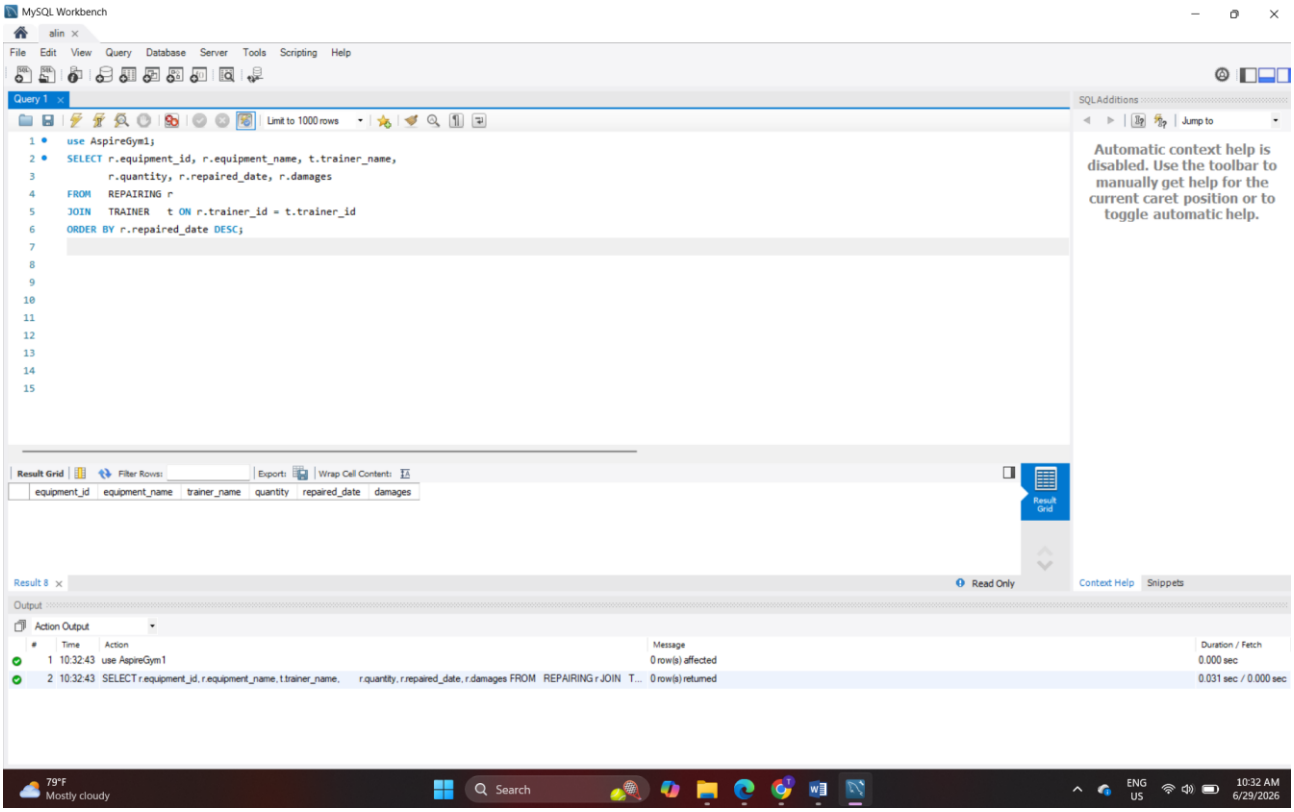


Figure 20: Screenshot showing Query 7 and its output in MySQL Workbench

Discussion / Critical Reflection

Looking back at this assignment, I am satisfied that all required tasks were completed successfully. The database for the Aspire Fitness Gym Management System was designed, normalized, implemented, and queried in a structured and methodical way.

One of the most valuable aspects of this assignment was reviewing and correcting the original ERD. Upon careful analysis, several issues were identified in the initial diagram: date fields were incorrectly typed as integers, some entities contained redundant duplicate attributes such as client_name appearing in multiple tables, and relationship lines did not consistently apply crow's foot cardinality notation. Addressing these issues required going back to the business requirements and re-thinking how entities relate to one another. This process deepened my understanding of what a well-normalized relational database should look like.

The normalization process itself proved more involved than anticipated. It was tempting at first to include all information within a single large table, but applying First, Second, and Third Normal Form clearly demonstrated why this leads to data redundancy and update anomalies. Separating trainer data from schedule data, and client data from subscription and payment data, produced a cleaner and more maintainable design.

MySQL Workbench was straightforward to work with once I became familiar with the interface. The visual schema designer was particularly useful for verifying that foreign key relationships were correctly set up. A small initial challenge was the correct use of the REFERENCES keyword for foreign keys and ensuring that parent tables were always created before their dependent child tables.

Writing the SQL queries was the most engaging part of the assignment. Moving beyond simple SELECT statements to use JOINs, GROUP BY, HAVING, and ORDER BY to address real business questions was rewarding. The payment aggregation query (Query 6) was especially instructive, as it illustrated how aggregate functions can be combined with filtering conditions to produce meaningful business intelligence.

If I were to approach this assignment again, I would begin with a more thorough reading of all the business rules before drawing any diagram, as this would have saved time during the correction phase. I would also plan the order of table creation more carefully from the outset to avoid foreign key dependency issues during implementation.

This assignment has significantly strengthened my understanding of relational database design and has motivated me to explore more advanced SQL features such as stored procedures, views, and indexing strategies in future studies.

Conclusion

This assignment required the design and implementation of a fully functional relational database for the Aspire Fitness Gym Management System. The work followed a structured process: beginning with business requirements analysis, progressing through conceptual and physical design, and concluding with a working SQL implementation in MySQL Workbench.

All four tasks were completed successfully:

- Task 1 identified ten key entities and produced a corrected, fully notated Entity Relationship Diagram using Visual Paradigm Community Edition, addressing data type errors and relationship notation issues found in the original design.
- Task 2 delivered fully normalized skeleton tables for all ten entities and comprehensive data dictionaries covering field names, data types, constraints, and descriptions.
- Task 3 implemented all ten tables in MySQL Workbench using correctly structured CREATE TABLE statements with appropriate primary and foreign key constraints, followed by sample INSERT statements populating each table with realistic data.
- Task 4 designed and implemented seven SQL queries addressing genuine business needs, ranging from simple directory listings to JOIN-based audit reports and aggregate financial summaries using GROUP BY and HAVING.

This work has comprehensively addressed the learning outcomes of the CIS095 module, applying problem-solving skills to design, develop, and implement a database system grounded in a real-world business context. It has also provided practical experience with MySQL Workbench as a professional database tool, and has reinforced the importance of rigorous normalization and careful ERD design before implementation begins.

Appendix

Normalization

Unnormalized Form (UNF)

The starting point was an unnormalized flat record representing a member's full data in a single row, including repeating groups:

client_id	client_name	trainer_name	session	equipment_name	subscription_status	payment_amt
1001	Ram Sharma	Anish Gurung	Yoga, Pilates	Mat, Block	Active	50, 60
1002	Sita Thapa	Mina Shrestha	Zumba	Speaker	Active	50

First Normalization Form (1NF) – Remove Repeating Groups

Each repeating group is expanded into separate rows so that all attributes are atomic:

client_id	client_name	trainer_name	session	equipment_name	subscription_status	payment_amt
1001	Ram Sharma	Anish Gurung	Yoga	Mat	Active	50
1001	Ram Sharma	Anish Gurung	Pilates	Block	Active	60
1002	Sita Thapa	Mina Shrestha	Zumba	Speaker	Active	50

Second Normalization Form (2NF) – Remove Partial Dependencies

Attributes depending only on part of the composite key are moved to separate tables:

CLIENT_MEMBER:

client_id (PK)	client_name	Contact
1001	Ram Sharma	984100100
1002	Sita Thapa	984100200

TRAINER:

trainer_id (PK)	trainer_name	Session
501	Anish Gurung	Pilates
502	Mina Shrestha	Zumba

Third Normalization Form (3NF) – Remove Transitive Dependencies

All non-key attributes now depend directly and solely on the primary key of their respective tables. No transitive dependencies remain. The final 3NF tables are:

- CLIENT_MEMBER (client_id PK, client_name, gender, age, contact, email, date_of_birth)
- TRAINER (trainer_id PK, trainer_name, salary, email, password)
- SCHEDULE (sched_id PK, trainer_id FK, equipment_id FK, session, week, day, time_start)
- EQUIPMENTS (equipment_id PK, equipment_name, date, quantity)
- REPAIRING (equipment_id PK, equipment_name, trainer_id FK, quantity, repaired_date, damages)
- SUBSCRIPTION (subscription_id PK, client_id FK, client_name, date, status)
- PAYMENT (payment_id PK, client_id FK, client_name, date, amount)
- TRANSACTION_RECORDS (trans_id PK, client_id FK, amount, date)
- REPORTS (report_id PK, trainer_id FK, client_name, client_id FK, equipment_id FK, trans_id FK, amount, date)
- INSURANCE (insurance_id PK, client_id FK, client_name, insurance_company, coverage_type, start_date, end_date, premium_amt, created_at, updated_at)

Primary Key and Foreign Key Reference Table

Table 12: Primary Key and Foreign Key Reference Table

Table	Primary Key	Foreign Keys
CLIENT_MEMBER	client_id	None
TRAINER	trainer_id	None
EQUIPMENTS	equipment_id	None
TRANSACTION_RECORDS	trans_id	None
SCHEDULE	sched_id	trainer_id → TRAINER, equipment_id → EQUIPMENTS
REPAIRING	equipment_id	trainer_id → TRAINER, equipment_id → EQUIPMENTS
SUBSCRIPTION	subscription_id	client_id → CLIENT_MEMBER
PAYMENT	payment_id	client_id → CLIENT_MEMBER
INSURANCE	insurance_id	client_id → CLIENT_MEMBER
REPORTS	report_id	trainer_id → TRAINER, client_id → CLIENT_MEMBER, equipment_id → EQUIPMENTS, trans_id → TRANSACTION_RECORDS